

UNIVERSITY OF ENGINEERING AND TECHNOLOGY PESHAWAR, PAKISTAN

Main Campus

DATA STRUCTURES

Lab 03 Assignment

Submitted By

Name: ANAS KHAN

Registration No: Bf25pwcs1691

Semester: 2nd Semester, BS Computer Science

Section: B

Submission Date: 30-09-2026

Submitted To: Sir Mudasir

DEPARTMENT OF COMPUTER SCIENCE

UNIVERSITY OF ENGINEERING AND TECHNOLOGY, PESHAWAR, PAKISTAN

Exercises 1: Memory Foundations

Exercises 1

Task 1:

Create one global integer and two local integers. Print their values and addresses. Add comments explaining scope and lifetime.

Code:

```
Lab_3 > C++ Ques2.cpp > task1()
1 // Anas Khan
2 // Registration Number: 25pwcs1691
3 // Section: B
4 #include <iostream>
5 using namespace std;
6
7 // Global variable
8 // Scope: The entire program/file after its declaration.
9 // Lifetime: Exists from program start until the program ends.
10 int globalNumber = 100;
11
12 void task1()
13 {
14     // Local variables
15     // Scope: Only inside this function.
16     // Lifetime: Created when the function is called and destroyed
17     // when the function finishes.
18     int localNumber1 = 200;
19     int localNumber2 = 300;
20
21     cout << "Global variable:" << endl;
22     cout << "Value: " << globalNumber << endl;
23     cout << "Address: " << &globalNumber << endl;
24
25     cout << "\nLocal variable 1:" << endl;
26     cout << "Value: " << localNumber1 << endl;
27     cout << "Address: " << &localNumber1 << endl;
28
29     cout << "\nLocal variable 2:" << endl;
30     cout << "Value: " << localNumber2 << endl;
31     cout << "Address: " << &localNumber2 << endl;
32 }
33
34 int main()
35 {
36     task1();
37 }
```

Output:

```
16 // Lifetime: Created when the function is called and destroyed
17 // when the function finishes.
...
Problems Output Debug Console Terminal Ports
PS D:\Data\Structures> cd "d:\Data\Structures\Lab_3\" ; if ($?) { g++ Ques2.cpp -o Ques2 } ; if ($?) { .\Ques2 }
Address: 0x404004

Local variable 1:
Value: 200
Address: 0x61ff0c

Local variable 2:
Value: 300
Address: 0x61ff08
PS D:\Data\Structures\Lab_3>
```

Task 2:

Declare a fixed array of five integers and a dynamic array of five integers. Print the address of every element in both arrays.

Code:

```
lab_3 > C++ Ques3.cpp > main()
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      // Fixed (static) array
7      // The array has a fixed size of 5.
8      // Its memory is automatically managed.
9      int fixedArray[5] = {10, 20, 30, 40, 50};
10
11     // Dynamic array
12     // Memory is allocated at runtime using new.
13     // It must be released using delete[].
14     int* dynamicArray = new int[5];
15
16     for (int i = 0; i < 5; i++)
17     {
18         dynamicArray[i] = (i + 1) * 10;
19     }
20
21     cout << "Addresses of fixed array elements:" << endl;
22
23     for (int i = 0; i < 5; i++)
24     {
25         cout << "fixedArray[" << i << "] = "
26             << &fixedArray[i] << endl;
27     }
28
29     cout << "\nAddresses of dynamic array elements:" << endl;
30
31     for (int i = 0; i < 5; i++)
32     {
33         cout << "dynamicArray[" << i << "] = "
34             << &dynamicArray[i] << endl;
35     }
36
37     // Release dynamically allocated memory.
38     delete[] dynamicArray;
39
40     return 0;
41 }
```

Output:

```
PS D:\Data\Structures\Lab_3> cd "d:\Data\Structures\Lab_3\" ; if ($?) { g++ Ques3.cpp -o Ques3 } ; if ($?) { .\Ques3 }
Addresses of fixed array elements:
fixedArray[0] = 0x61fec0
fixedArray[1] = 0x61fec4
fixedArray[2] = 0x61fec8
fixedArray[3] = 0x61fecC
fixedArray[4] = 0x61fed0

Addresses of dynamic array elements:
dynamicArray[0] = 0x10c1858
dynamicArray[1] = 0x10c185c
dynamicArray[2] = 0x10c1860
dynamicArray[3] = 0x10c1864
dynamicArray[4] = 0x10c1868
PS D:\Data\Structures\Lab_3>
```

Task 3:

Write a function with a local variable. Call the function three times and print the local variable address each time. Record what you observe.

Code:

```
C++ Ques2.cpp u C++ Ques3.cpp u C++ Ques4.cpp u X
Lab_3 > C++ Ques4.cpp > main()
1 #include <iostream>
2 using namespace std;
3
4 void showAddress()
5 {
6     // This variable is local to showAddress().
7     // Its lifetime begins when the function is called
8     // and ends when the function returns.
9     int localVariable = 50;
10
11     cout << "Local variable address: "
12         << &localVariable << endl;
13 }
14
15 int main()
16 {
17     cout << "First function call:" << endl;
18     showAddress();
19
20     cout << "\nSecond function call:" << endl;
21     showAddress();
22
23     cout << "\nThird function call:" << endl;
24     showAddress();
25
26     return 0;
27 }
```

Output:

```
27 }
Problems Output Debug Console Terminal Ports
● PS D:\Data\Structures\Lab_3> cd "d:\Data\Structures\Lab_3\" ; if ($?) { g++ Ques4.cpp -o Ques4 } ; if ($?) { .\Ques4 }
First function call:
Local variable address: 0x61feec

Second function call:
Local variable address: 0x61feec

Third function call:
Local variable address: 0x61feec
○ PS D:\Data\Structures\Lab_3>
```

Task 4:

Create a dynamic array and intentionally comment out delete [] for one test run. Explain why this is bad practice even if the operating system eventually reclaims memory after the process exits.

Code:

```
Lab_3 > C++ Ques5.cpp > main()
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      // Allocate an array of 5 integers dynamically.
7      int* numbers = new int[5];
8
9      for (int i = 0; i < 5; i++)
10     {
11         numbers[i] = i + 1;
12     }
13
14     cout << "Dynamic array values:" << endl;
15
16     for (int i = 0; i < 5; i++)
17     {
18         cout << numbers[i] << " ";
19     }
20
21     cout << endl;
22
23     /*
24     Intentionally commented out for demonstration:
25     delete[] numbers;
26
27     Without delete[], the dynamically allocated memory
28     is not released while the program is running.
29     */
30
31     return 0;
32 }
33
```

Output:

```
Problems Output Debug Console Terminal Ports
PS D:\Data(Structures)\Lab_3> cd "d:\Data(Structures)\Lab_3\" ; if ($?) { g++ Ques5.cpp -o Ques5 } ; if ($?) { .\Ques5 }
1 2 3 4 5
PS D:\Data(Structures)\Lab_3>
```

Task 5:

Write a short paragraph comparing scope, lifetime, and storage for global variables, local variables, fixed arrays, and dynamic arrays.

Explanation:

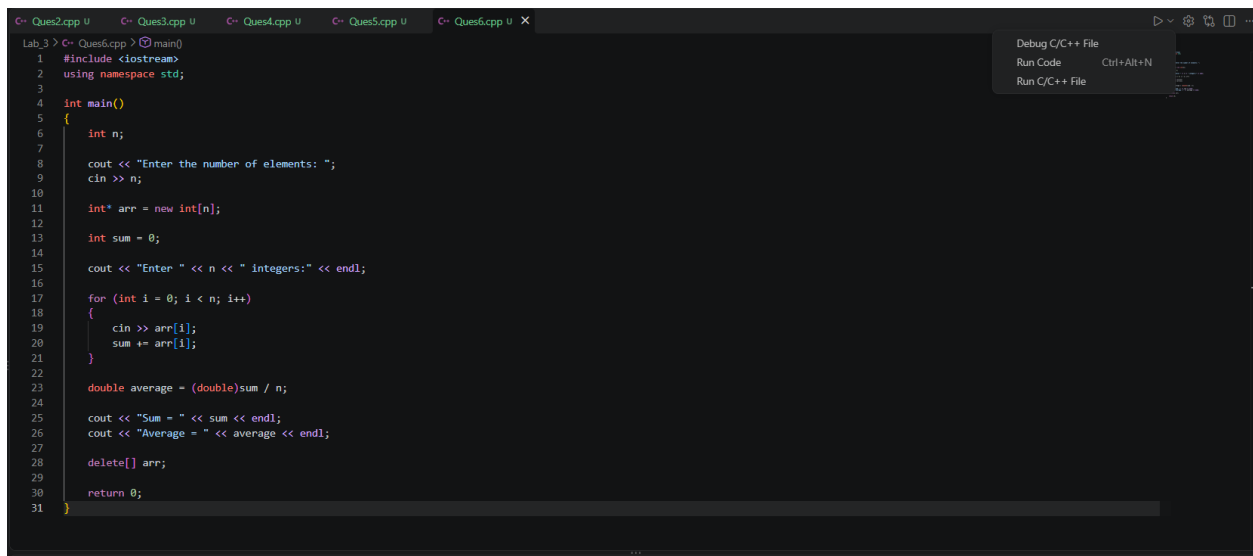
Global variables have program-wide scope after their declaration and remain alive for the entire duration of the program. They are stored in static storage and are created before main() begins. Local variables have block or function scope and normally use automatic storage, meaning they are created when execution reaches their declaration and destroyed when their block or function ends. A fixed array declared locally is also an automatic object and stores all of its elements together in its allocated storage, while a global fixed array has static storage duration. A dynamic array is different because its memory is allocated from the free store (heap) at runtime using new[]; the pointer used to access it may be local, but the dynamically allocated array remains alive until delete[] is used. Thus, **scope describes where a variable can be accessed, lifetime describes how long it exists, and storage describes where/how its memory is managed.**

Exercises 2

Task 1:

Ask the user for n, create a dynamic array of n integers, and display the sum and average.

Code:



```
Lab_3 > C++ Ques6.cpp > main()
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      int n;
7
8      cout << "Enter the number of elements: ";
9      cin >> n;
10
11     int* arr = new int[n];
12
13     int sum = 0;
14
15     cout << "Enter " << n << " integers:" << endl;
16
17     for (int i = 0; i < n; i++)
18     {
19         cin >> arr[i];
20         sum += arr[i];
21     }
22
23     double average = (double)sum / n;
24
25     cout << "Sum = " << sum << endl;
26     cout << "Average = " << average << endl;
27
28     delete[] arr;
29
30     return 0;
31 }
```

Output:



```
PS D:\Data\Structures\Lab_3> cd "d:\Data\Structures\Lab_3\" ; if ($?) { g++ Ques6.cpp -o Ques6 } ; if ($?) { .\Ques6 }
Enter the number of elements: 4
Enter 4 integers:
1
2
3
4
Sum = 10
Average = 2.5
PS D:\Data\Structures\Lab_3>
```

Task 2:

Create a dynamic array and find the maximum and minimum values without using STL algorithms.

Code:

```
Ques2.cpp U C- Ques3.cpp U C- Ques4.cpp U C- Ques5.cpp U C- Ques6.cpp U C- Ques7.cpp U X
1 #include <iostream>
2 using namespace std;
3
4 int main()
5 {
6     int n;
7
8     cout << "Enter the number of elements: ";
9     cin >> n;
10
11     int* arr = new int[n];
12
13     cout << "Enter " << n << " integers:" << endl;
14
15     for (int i = 0; i < n; i++)
16     {
17         cin >> arr[i];
18     }
19
20     int maximum = arr[0];
21     int minimum = arr[0];
22
23     // Find maximum and minimum without STL algorithms
24     for (int i = 1; i < n; i++)
25     {
26         if (arr[i] > maximum)
27         {
28             maximum = arr[i];
29         }
30
31         if (arr[i] < minimum)
32         {
33             minimum = arr[i];
34         }
35     }
36
37     cout << "Maximum = " << maximum << endl;
38     cout << "Minimum = " << minimum << endl;
39
40     delete[] arr;
41
42     return 0;
43 }
```

Output:

```
Problems Output Debug Console Terminal Ports
PS D:\Data\Structures\Lab_3> cd "d:\Data\Structures\Lab_3\" ; if ($?) { g++ Ques7.cpp -o Ques7 } ; if ($?) { .\Ques7 }
Enter the number of elements: 5
Enter 5 integers:
1
4
7
8
10
Maximum = 10
Minimum = 1
PS D:\Data\Structures\Lab_3>
```

Task 3:

Reverse a dynamic array in place using two indices (left and right).

Code:

```
Lab_3 > C++ Ques8.cpp > (main)
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      int n;
7
8      cout << "Enter the number of elements: ";
9      cin >> n;
10
11      int* arr = new int[n];
12
13      cout << "Enter " << n << " integers:" << endl;
14
15      for (int i = 0; i < n; i++)
16      {
17          cin >> arr[i];
18      }
19
20      // Two indices
21      int left = 0;
22      int right = n - 1;
23
24      // Reverse the array in place
25      while (left < right)
26      {
27          int temp = arr[left];
28          arr[left] = arr[right];
29          arr[right] = temp;
30
31          left++;
32          right--;
33      }
34
35      cout << "Reversed array: ";
36
37      for (int i = 0; i < n; i++)
38      {
39          cout << arr[i] << " ";
40      }
41
42      cout << endl;
43      delete[] arr;
44
45      return 0;
46
47 }
```

Output:

```
Problems  Output  Debug Console  Terminal  Ports
PS D:\Data\Structures\Lab_3> cd "d:\Data\Structures\Lab_3" ; if ($?) { g++ Ques8.cpp -o Ques8 } ; if ($?) { .\Ques8 }
Enter the number of elements: 6
Enter 6 integers:
1
2
3
4
5
6
Reversed array: 6 5 4 3 2 1
PS D:\Data\Structures\Lab_3>
```

Task 4:

Create an array of n student marks. Count how many marks are ≥ 50 and how many are < 50 .

Code:

```
Lab_3 > C++ Quest9.cpp > [main]
1 #include <iostream>
2 using namespace std;
3
4 int main()
5 {
6     int n;
7
8     cout << "Enter the number of students: ";
9     cin >> n;
10
11     int* marks = new int[n];
12
13     int passed = 0;
14     int failed = 0;
15
16     cout << "Enter the marks:" << endl;
17
18     for (int i = 0; i < n; i++)
19     {
20         cin >> marks[i];
21
22         if (marks[i] >= 50)
23         {
24             passed++;
25         }
26         else
27         {
28             failed++;
29         }
30     }
31
32     cout << "Marks >= 50: " << passed << endl;
33     cout << "Marks < 50: " << failed << endl;
34
35     delete[] marks;
36
37     return 0;
38 }
```

Output:

```
U 20 cin >> marks[i];
U
Problems Output Debug Console Terminal Ports
PS D:\Data\Structures\Lab_3> cd "d:\Data\Structures\Lab_3" ; if ($?) { g++ Quest9.cpp -o Quest9 } ; if ($?) { .\Quest9 }
Enter the number of students: 7
Enter the marks:
15
67
90
50
43
21
Marks >= 50: 3
Marks < 50: 4
PS D:\Data\Structures\Lab_3>
```

Task 5:

Expand a dynamic array from n elements to $n+3$ elements while preserving the original values.

Code:

```
Lab_3 > C- Quest10.cpp > (main)
1 #include <iostream>
2 using namespace std;
3
4 int main()
5 {
6     int n;
7
8     cout << "Enter the number of elements: ";
9     cin >> n;
10
11     int* arr = new int[n];
12
13     cout << "Enter " << n << " integers:" << endl;
14
15     for (int i = 0; i < n; i++)
16     {
17         cin >> arr[i];
18     }
19
20     // Create a new array with n + 3 elements
21     int newSize = n + 3;
22     int* newArr = new int[newSize];
23
24     // Copy original values
25     for (int i = 0; i < n; i++)
26     {
27         newArr[i] = arr[i];
28     }
29
30     // Give values to the three new elements
31     newArr[n] = 0;
32     newArr[n + 1] = 0;
33     newArr[n + 2] = 0;
34
35     // Delete old array
36     delete[] arr;
37
38     // Make arr point to the new array
39     arr = newArr;
40
41     cout << "Expanded array: ";
42
43     for (int i = 0; i < newSize; i++)
44     {
45         cout << arr[i] << " ";
46     }
47
48     cout << endl;
49
50     delete[] arr;
51
52     return 0;
53 }
```

Output:

```
27     newArr[i] = arr[i];
28 }
Problems Output Debug Console Terminal Ports
PS D:\DataStructures\Lab_3> cd "D:\DataStructures\Lab_3" ; if ($?) { g++ Quest10.cpp -o Quest10 }; if ($?) { .\Quest10 }
Enter the number of elements: 5
Enter 5 integers:
1
2
3
4
5
Expanded array: 1 2 3 4 5 0 0 0
PS D:\DataStructures\Lab_3>
```

Task 6:

Write a function `int* createArray(int n)` that returns a newly allocated array, and another function `void destroyArray(int*& p)` that deletes it and sets the pointer to `nullptr`.

Code:

```
Lab_3 > C:\Quest11.cpp > (main)
1 #include <iostream>
2 using namespace std;
3
4 // Creates and returns a dynamic array
5 int* createArray(int n)
6 {
7     int* p = new int[n];
8
9     return p;
10 }
11
12 // Deletes the array and sets the pointer to nullptr
13 void destroyArray(int*& p)
14 {
15     delete[] p;
16     p = nullptr;
17 }
18
19 int main()
20 {
21     int n;
22
23     cout << "Enter the size of the array: ";
24     cin >> n;
25
26     int* arr = createArray(n);
27
28     cout << "Enter " << n << " integers:" << endl;
29
30     for (int i = 0; i < n; i++)
31     {
32         cin >> arr[i];
33     }
34
35     cout << "Array: ";
36
37     for (int i = 0; i < n; i++)
38     {
39         cout << arr[i] << " ";
40     }
41
42     cout << endl;
43
44     destroyArray(arr);
45
46     if (arr == nullptr)
47     {
48         cout << "Array destroyed successfully." << endl;
49     }
50
51     return 0;
52 }
```

Output:

```
26     int* arr = createArray(n);
27
28     cout << "Enter " << n << " integers:" << endl;
29
30     for (int i = 0; i < n; i++)
31     {
32         cin >> arr[i];
33     }
34
35     cout << "Array: ";
36
37     for (int i = 0; i < n; i++)
38     {
39         cout << arr[i] << " ";
40     }
41
42     cout << endl;
43
44     destroyArray(arr);
45
46     if (arr == nullptr)
47     {
48         cout << "Array destroyed successfully." << endl;
49     }
50
51     return 0;
52 }
```

Problems Output Debug Console Terminal Ports

PS D:\Data\Structures\Lab_3> cd "d:\Data\Structures\Lab_3\" ; if (\$?) { g++ Quest11.cpp -o Quest11 } ; if (\$?) { .\Quest11 }

Enter the size of the array: 4

Enter 4 integers:

1

2

3

7

Array: 1 2 3 7

Array destroyed successfully.

PS D:\Data\Structures\Lab_3>

Task 7:

Challenge: Implement a small “score bag” with capacity and used-size variables. When used-size reaches capacity, allocate double capacity and copy the elements.

Code:

```
1 // Lab 3: Question 12
2 #include <iostream>
3 using namespace std;
4
5 int main()
6 {
7     int capacity = 2;
8     int used = 0;
9
10    int* scores = new int[capacity];
11
12    int score;
13    cout << "Enter scores (-1 to stop): " << endl;
14    while (true)
15    {
16        cin >> score;
17
18        if (score == -1)
19        {
20            break;
21        }
22
23        // If array is full, double the capacity
24        if (used == capacity)
25        {
26            int newCapacity = capacity * 2;
27            int* newScores = new int[newCapacity];
28
29            // Copy existing scores
30            for (int i = 0; i < used; i++)
31            {
32                newScores[i] = scores[i];
33            }
34
35            // Delete old array
36            delete[] scores;
37
38            // Point to new array
39            scores = newScores;
40
41            // Update capacity
42            capacity = newCapacity;
43            cout << "Capacity expanded to "
44                  << capacity << endl;
45        }
46
47        // Add new score
48        scores[used] = score;
49        used++;
50    }
51
52    cout << "Scores: ";
53    for (int i = 0; i < used; i++)
54    {
55        cout << scores[i] << " ";
56    }
57    cout << endl;
58
59    cout << "Used size: " << used << endl;
60    cout << "Capacity: " << capacity << endl;
61
62    // Free memory
63    delete[] scores;
64    return 0;
65 }
```

Output:

```
1 PS D:\Data\Structures\Lab_3> cd "D:\Data\Structures\Lab_3" ; if ($?) { g++ Ques12.cpp -o Ques12 } ; if ($?) { .\Ques12 }
2 Enter scores (-1 to stop):
3 1
4 2
5 3
6 Capacity expanded to 4
7 4
8 5
9 Capacity expanded to 8
10 6
11 7
12 8
13 9
14 Capacity expanded to 16
15 -1
16
17 Scores: 1 2 3 4 5 6 7 8 9
18 Used size: 9
19 Capacity: 16
20 PS D:\Data\Structures\Lab_3>
```

Exercises 3

Task 1:

Search for a student roll number in an unsorted integer array.

Code:

```
Lab_3 > C++ Ques13.cpp > (main)
1 #include <iostream>
2 using namespace std;
3
4 int main()
5 {
6     int rollNumbers[] = {105, 102, 108, 101, 110, 103};
7     int size = 6;
8
9     int target;
10
11     cout << "Enter student roll number to search: ";
12     cin >> target;
13
14     int index = -1;
15
16     // Sequential search
17     for (int i = 0; i < size; i++)
18     {
19         if (rollNumbers[i] == target)
20         {
21             index = i;
22             break;
23         }
24     }
25
26     if (index != -1)
27     {
28         cout << "Roll number found at index: " << index << endl;
29     }
30     else
31     {
32         cout << "Roll number not found." << endl;
33     }
34
35     return 0;
36 }
```

Output:

```
17 | for (int i = 0; i < size; i++)
Problems Output Debug Console Terminal Ports
PS D:\Data\Structures\Lab_3> cd "d:\Data\Structures\Lab_3\" ; if ($?) { g++ Ques13.cpp -o Ques13 } ; if ($?) { .\Ques13 }
Enter student roll number to search: 110
Roll number found at index: 4
PS D:\Data\Structures\Lab_3> cd "d:\Data\Structures\Lab_3\" ; if ($?) { g++ Ques13.cpp -o Ques13 } ; if ($?) { .\Ques13 }
Enter student roll number to search: 120
Roll number not found.
PS D:\Data\Structures\Lab_3> |
```

Task 2:

Modify linear search to count all occurrences of a target value instead of stopping at the first match.

Code:

```
> C++ Ques14.cpp > main()
#include <iostream>
using namespace std;

int main()
{
    int numbers[] = {10, 20, 10, 30, 10, 40, 20};
    int size = 7;

    int target;

    cout << "Enter target value: ";
    cin >> target;

    int count = 0;

    for (int i = 0; i < size; i++)
    {
        if (numbers[i] == target)
        {
            count++;
        }
    }

    cout << "Number of occurrences: " << count << endl;

    return 0;
}
```

Output:

```
PS D:\Data\Structures\Lab_3> cd "d:\Data\Structures\Lab_3\" ; if ($?) { g++ Ques14.cpp -o Ques14 } ; if ($?) { .\Ques14 }
Enter target value: 10
Number of occurrences: 3
PS D:\Data\Structures\Lab_3> cd "d:\Data\Structures\Lab_3\" ; if ($?) { g++ Ques14.cpp -o Ques14 } ; if ($?) { .\Ques14 }
Enter target value: 50
Number of occurrences: 0
PS D:\Data\Structures\Lab_3> cd "d:\Data\Structures\Lab_3\" ; if ($?) { g++ Ques14.cpp -o Ques14 } ; if ($?) { .\Ques14 }
Enter target value: 40
Number of occurrences: 1
PS D:\Data\Structures\Lab_3>
```

Task 3:

Return the index of the last occurrence of a target.

Code:

```
Lab_3 > C++ Ques15.cpp > main()
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      int numbers[] = {10, 20, 30, 20, 40, 20, 50};
7      int size = 7;
8
9      int target;
10
11     cout << "Enter target value: ";
12     cin >> target;
13
14     int lastIndex = -1;
15
16     for (int i = 0; i < size; i++)
17     {
18         if (numbers[i] == target)
19         {
20             lastIndex = i;
21         }
22     }
23
24     if (lastIndex != -1)
25     {
26         cout << "Last occurrence found at index: "
27             << lastIndex << endl;
28     }
29     else
30     {
31         cout << "Target not found." << endl;
32     }
33
34     return 0;
35 }
```

Output:

```

30  {
31      cout << "Target not found." << endl;
32  }
33  }
34  }
35  }

Problems Output Debug Console Terminal Ports
● PS D:\Data\Structures\Lab_3> cd "d:\Data\Structures\Lab_3\" ; if ($?) { g++ Ques15.cpp -o Ques15 } ; if ($?) { .\Ques15 }
Enter target value: 50
Last occurrence found at index: 6
○ PS D:\Data\Structures\Lab_3>
```

Task 4:

Perform linear search on a dynamic array created from user input.

Code:

```
Lab3 > C++ Quest16pp > @main
1 #include <iostream>
2 using namespace std;
3
4 int main()
5 {
6     int n;
7
8     cout << "Enter the number of elements: ";
9     cin >> n;
10
11     int* arr = new int[n];
12
13     cout << "Enter " << n << " integers:" << endl;
14
15     for (int i = 0; i < n; i++)
16     {
17         cin >> arr[i];
18     }
19
20     int target;
21
22     cout << "Enter value to search: ";
23     cin >> target;
24
25     int index = -1;
26
27     // Linear search
28     for (int i = 0; i < n; i++)
29     {
30         if (arr[i] == target)
31         {
32             index = i;
33             break;
34         }
35     }
36
37     if (index != -1)
38     {
39         cout << "Target found at index: " << index << endl;
40     }
41     else
42     {
43         cout << "Target not found." << endl;
44     }
45
46     delete[] arr;
47
48     return 0;
49 }
```

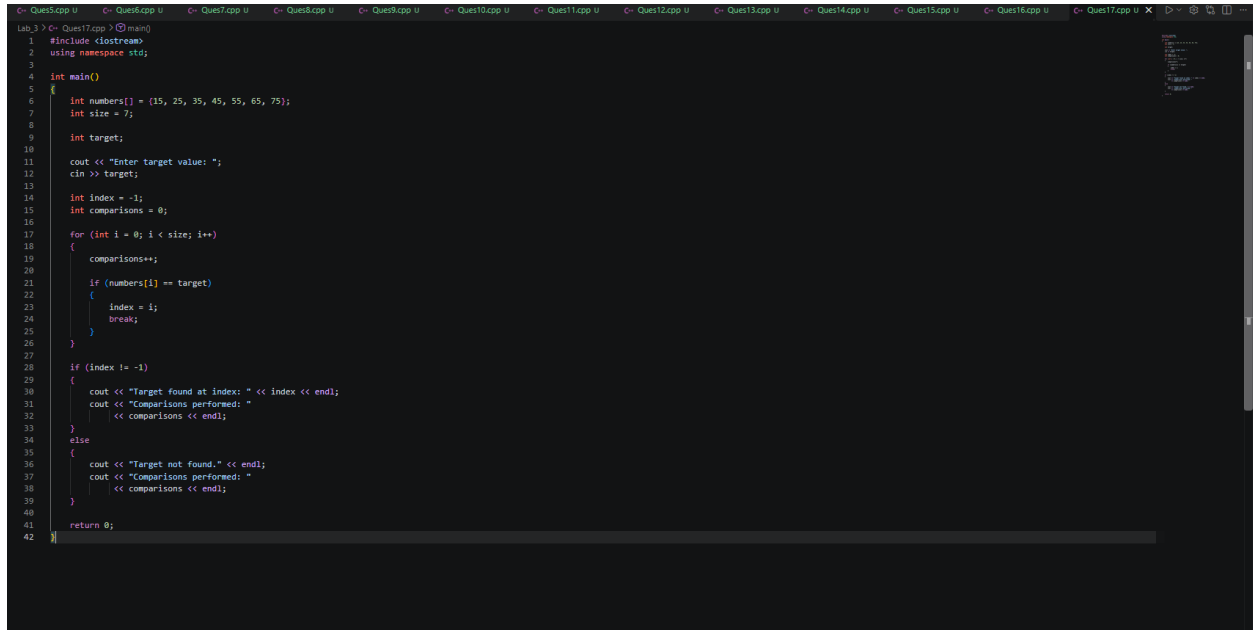
Output:

```
U 15 for (int i = 0; i < n; i++)
U
U Problems Output Debug Console Terminal Ports
U PS D:\Data\Structures\Lab_3> cd "d:\Data\Structures\Lab_3\" ; if ($?) { g++ Quest16.cpp -o Quest16 } ; if ($?) { .\Quest16 }
U Enter the number of elements: 5
U Enter 5 integers:
U 10
U 20
U 30
U 35
U 37
U Enter value to search: 35
U Target found at index: 3
U PS D:\Data\Structures\Lab_3>
U
```

Task 5:

Count the number of comparisons performed before the target is found.

Code:



```
1 #include <iostream>
2 using namespace std;
3
4 int main()
5 {
6     int numbers[] = {15, 25, 35, 45, 55, 65, 75};
7     int size = 7;
8
9     int target;
10
11     cout << "Enter target value: ";
12     cin >> target;
13
14     int index = -1;
15     int comparisons = 0;
16
17     for (int i = 0; i < size; i++)
18     {
19         comparisons++;
20
21         if (numbers[i] == target)
22         {
23             index = i;
24             break;
25         }
26     }
27
28     if (index != -1)
29     {
30         cout << "Target found at index: " << index << endl;
31         cout << "Comparisons performed: "
32              << comparisons << endl;
33     }
34     else
35     {
36         cout << "Target not found." << endl;
37         cout << "Comparisons performed: "
38              << comparisons << endl;
39     }
40
41     return 0;
42 }
```

Output:



```
PS D:\DataStructures\Lab_3> cd "d:\DataStructures\Lab_3" ; if ($?) { g++ Ques17.cpp -o Ques17 } ; if ($?) { .\Ques17 }
Enter target value: 45
Target found at index: 3
Comparisons performed: 4
PS D:\DataStructures\Lab_3>
```

Task 6:

Search for a target inside a 2D matrix and print the row and column where it is found.

Code:

```
h3_3> C++ Quest18.cpp [main]
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      int matrix[3][4] =
7      {
8          {10, 20, 30, 40},
9          {50, 60, 70, 80},
10         {90, 100, 110, 120}
11     };
12
13     int target;
14
15     cout << "Enter target value: ";
16     cin >> target;
17
18     bool found = false;
19
20     // Search row by row
21     for (int row = 0; row < 3; row++)
22     {
23         for (int col = 0; col < 4; col++)
24         {
25             if (matrix[row][col] == target)
26             {
27                 cout << "Target found at row "
28                     << row << ", column "
29                     << col << endl;
30                 found = true;
31                 break;
32             }
33         }
34
35         if (found)
36         {
37             break;
38         }
39     }
40
41     if (!found)
42     {
43         cout << "Target not found." << endl;
44     }
45
46     return 0;
47 }
48
```

Output:

```
U 26 {
U 27     cout << "Target found at row "
U 28     << row << ", column "
U
U Problems Output Debug Console Terminal Ports
U PS D:\Data\Structures\Lab_3> cd "d:\Data\Structures\Lab_3" ; if ($?) { g++ Quest18.cpp -o Quest18 } ; if ($?) { .\Quest18 }
U Enter target value: 110
U Target found at row 2, column 2
U PS D:\Data\Structures\Lab_3> cd "d:\Data\Structures\Lab_3" ; if ($?) { g++ Quest18.cpp -o Quest18 } ; if ($?) { .\Quest18 }
U Enter target value: 220
U Target not found.
U PS D:\Data\Structures\Lab_3> |
```

Task 7:

Challenge: Search a list of marks and return all indices where the target mark occurs.

Code:

```
Lab_3 > C:\Ques19.cpp > main()
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      int marks[] = {75, 60, 50, 75, 90, 75, 40, 60};
7      int size = 8;
8
9      int target;
10
11      cout << "Enter target mark: ";
12      cin >> target;
13
14      int count = 0;
15
16      cout << "Target found at indices: ";
17
18      for (int i = 0; i < size; i++)
19      {
20          if (marks[i] == target)
21          {
22              cout << i << " ";
23              count++;
24          }
25      }
26
27      if (count == 0)
28      {
29          cout << "None";
30      }
31
32      cout << endl;
33
34      cout << "Total occurrences: " << count << endl;
35
36      return 0;
37 }
```

Output:

```
PS D:\Data\Structures\Lab_3> cd "d:\Data\Structures\Lab_3\" ; if ($?) { g++ Ques19.cpp -o Ques19 } ; if ($?) { .\Ques19 }
Enter target mark: 100
Target found at indices: None
Total occurrences: 0
PS D:\Data\Structures\Lab_3> cd "d:\Data\Structures\Lab_3\" ; if ($?) { g++ Ques19.cpp -o Ques19 } ; if ($?) { .\Ques19 }
Enter target mark: 76
Target found at indices: None
Total occurrences: 0
PS D:\Data\Structures\Lab_3> cd "d:\Data\Structures\Lab_3\" ; if ($?) { g++ Ques19.cpp -o Ques19 } ; if ($?) { .\Ques19 }
Enter target mark: 75
Target found at indices: 0 3 5
Total occurrences: 3
PS D:\Data\Structures\Lab_3>
```